UNIVERSITÉ LAVAL

TP1 : construire, manipuler et dessiner un réseau

Avant de débuter

`igraph` est installé sur le serveur. Sur votre ordinateur : `install.packages("igraph")`.

```
library(tidyverse)
library(igraph)   # APRÈS le tidyverse
```

Lisez les messages de chargement : `igraph` **masque** plusieurs fonctions du tidyverse (`as_data_frame()`, `groups()`, `union()`, `crossing()`, `simplify()`…). Dans ce TP, si vous avez besoin de la version du tidyverse, écrivez `tibble::as_data_frame()` ou `dplyr::union()`.

Les données

Game of Thrones, saison 1 : deux personnages sont liés quand ils interagissent (l'un parle juste après l'autre, parle de l'autre, ils apparaissent dans la même scène…). Données d'A. Beveridge (*Network of Thrones*).

Fichier	Contenu
`/home/public/reseaux/got_edges.csv`	une ligne par lien : `from`, `to`, `weight` (nombre d'interactions)
`/home/public/reseaux/got_sex.csv`	une ligne par personnage : `name`, `sex` (`F` ou `M`)

Rappel : les fonctions

```
graph_from_data_frame(aretes, directed = ..., vertices = ...)   # liste d'arêtes → graphe
vcount(g); ecount(g)                      # nombre de nœuds, de liens
V(g); E(g); V(g)$attribut; E(g)$weight    # nœuds, liens, attributs
vertex_attr(g); edge_attr(g)              # tous les attributs, sous forme de liste
degree(g); strength(g)                    # nombre de liens ; somme des poids
as_adjacency_matrix(g, sparse = FALSE)                  # matrice d'adjacence (0/1)
as_adjacency_matrix(g, attr = "weight", sparse = FALSE) # matrice des poids
plot(g, layout = ..., vertex.color = ..., vertex.size = ..., vertex.label = NA)   # ?igra
```

À votre tour

0. Importez les deux fichiers. Combien de lignes dans chacun ? Combien de personnages **distincts** apparaissent dans la liste d'arêtes (colonnes `from` **et** `to`) ? Est-ce cohérent avec le fichier des sexes ?
1. Construisez un graphe avec `graph_from_data_frame(got_aretes)`, **sans** préciser `directed`. Lisez l'en-tête : que signifient les lettres et les deux nombres ? Reconstruisez-le **non dirigé**, avec `vertices = got_sexe`, sous le nom `got`. Vérifiez avec `vcount()` et `ecount()`.
2. Listez les attributs des nœuds et des liens avec `vertex_attr()` et `edge_attr()`. Combien de personnages de chaque sexe ? Quel est le lien le plus **lourd** (`which.max()` sur `E(got)$weight`) ? Entre qui, et quel poids ?
3. Dessinez `got` : nœuds colorés selon le sexe, taille selon le degré, **sans** étiquettes. Fixez la disposition avec `set.seed()` et `layout = layout_with_fr`. Dessinez-le une seconde fois avec une autre disposition (`layout_in_circle` ou `layout_with_kk`), côte à côte (`par(mfrow = c(1, 2))`). Qu'est-ce qui change ? Qu'est-ce qui ne change pas ?
4. Calculez la matrice d'adjacence de `got` (`sparse = FALSE`). Quelle est sa dimension ? Est-elle symétrique (`isSymmetric()`) ? La somme de chaque ligne est-elle égale à `degree(got)` ? Recommencez avec `attr = "weight"` : à quoi est égale la somme des lignes ? Vérifiez avec `strength()`. Que vaut la case `["Ned", "Robert"]` ?
5. Recalculez le degré **sans igraph** : `pivot_longer()` sur `from` et `to` de `got_aretes`, puis `count()`. Comparez avec `degree(got)` pour tous les personnages (attention à l'ordre des noms).

Et si… ?

Pour chaque question, **prédisez d'abord la réponse sans exécuter**, puis vérifiez dans R.

1. Et si on gardait le graphe **dirigé** de la question 1 ? Que valent `degree(g, "Ned", mode = "in")` et `mode = "out"` ? Ces deux nombres ont-ils un sens pour ces données ?
2. Et si `Ned` était écrit `Eddard` dans le fichier des sexes ? Et si, au lieu de `vertices`, on ajoutait le sexe avec `V(g)$sex <- got_sexe$sex` sur un graphe construit sans `vertices` ? Combien de personnages reçoivent le mauvais sexe ?
3. Et si on ajoutait au fichier des sexes un personnage **sans aucun lien** (`Ghost`, le loup de Jon) ? Combien de nœuds avec `vertices`, et sans ?
4. Et si on exécutait deux fois `plot(got, vertex.label = NA)` **sans** `set.seed()` ? Les deux images sont-elles identiques ? Lequel des deux dessins est le « vrai » ?
5. Et si on reconstruisait un graphe à partir de la matrice **pondérée** de la question 4 avec `graph_from_adjacency_matrix(..., mode = "undirected")`, **sans** `weighted = TRUE` ? Lisez l'en-tête : combien de liens ? Combien entre Ned et Robert (`count_multiple()`) ?

Pour aller plus loin : `ggraph`

`ggraph` dessine les réseaux avec la grammaire de `ggplot2` (installé sur le serveur ; sur votre ordinateur : `install.packages("ggraph")`).

```r
library(ggraph)
set.seed(1)
ggraph(got, layout = "fr") +
  geom_edge_link(aes(edge_width = weight), alpha = 0.2) +
  geom_node_point(aes(colour = sex, size = degree(got))) +
  geom_node_text(aes(label = if_else(degree(got) >= 30, name, "")), repel = TRUE) +
  scale_edge_width(range = c(0.2, 2)) +
  theme_void()
```

Qu'apporte la largeur des liens ? Essayez `layout = "kk"`, puis `facet_nodes(~ sex)`.